

CO2 AT2

N. RISHITHA

192425011

Question 1

Recommended Technique: Autoencoder (Unsupervised Neural Network)

Explanation

An Autoencoder is an unsupervised deep learning model used to learn the normal patterns of unlabeled data. In this scenario, the hospital has thousands of unlabeled ECG records, making manual labeling difficult.

The ECG signals are first preprocessed to remove noise and improve data quality. The Autoencoder is then trained using these ECG signals. It learns the normal heartbeat patterns by compressing the data into important features and reconstructing the original signal.

After reconstruction, the model calculates the reconstruction error. If the error is low, the ECG is considered normal. If the error is high, the ECG is identified as abnormal. This helps doctors quickly detect possible heart abnormalities without requiring labeled training data.

Advantages

- Works with unlabeled ECG data.
- Automatically learns normal ECG patterns.
- Detects abnormal heart signals accurately.
- Reduces manual effort and saves time.

DEEP NEURAL NETWORK ARCHITECTURE
(Autoencoder for ECG Abnormality Detection)

The diagram illustrates a Deep Neural Network (DNN) architecture designed for ECG Abnormality Detection, structured as an autoencoder. The architecture is divided into three main sections: the **ENCODER (Feature Learning)**, the **LATENT SPACE (Compressed Representation)**, and the **DECODER (Reconstruction)**.

ENCODER (Feature Learning): This section processes the input ECG signal (TD Time Series) into an input vector of n features. The input vector is fed into a series of hidden layers: Hidden Layer 1 (ReLU), Hidden Layer 2 (ReLU), and Hidden Layer 3 (ReLU). The number of neurons in these layers decreases from 128 to 64 to 32. The output of the encoder is a latent space representation consisting of k neurons, where $k \ll n$.

LATENT SPACE (Compressed Representation): The latent space is a compressed representation of the input features, consisting of k neurons.

DECODER (Reconstruction): The latent space representation is fed into a series of hidden layers: Hidden Layer 4 (ReLU), Hidden Layer 5 (ReLU), and Hidden Layer 6 (ReLU). The number of neurons in these layers increases from 32 to 64 to 128. The output of the decoder is a reconstructed ECG signal.

OUTPUT LAYER (Reconstructed ECG Signal): The reconstructed ECG signal is compared with the input ECG signal to calculate the reconstruction error, which is the Mean Squared Error (MSE) or Mean Absolute Error (MAE). The reconstruction error is calculated as $\frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$.

RECONSTRUCTION ERROR (MSE / MAE): The reconstruction error is used to classify the input as Normal or Abnormal. If the error is less than or equal to a threshold τ , the input is classified as Normal. If the error is greater than τ , the input is classified as Abnormal.

OUTPUT (Classification): The classification result is either Normal or Abnormal.

DETECTION RESULT: The final output of the system is the detection result, which is either Normal ECG (indicated by a green checkmark) or Abnormal ECG (indicated by a red warning triangle).

LEGEND:

- Input / Output Neuron
- Hidden Neuron
- Information Flow
- Weight Connection

WORKING PRINCIPLE

The autoencoder learns to reconstruct normal ECG signals with low error. Abnormal ECG signals produce higher reconstruction error, which is used to classify the input as Normal or Abnormal.

Patient_ID													
	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Patient_ID	Age	Gender	Heart_Rat	QRS_Dura	PR_Interv	QT_Interv	ST_Deviat	ECG_Featu	ECG_Featu	ECG_Featu	ECG_Featu	ECG_Featu4
2	P1000	65	Male	58	0.119	0.144	0.322	-0.149	0.741	0.545	0.59	0.032	
3	P1001	30	Male	84	0.1	0.123	0.332	0.125	0.545	0.22	0.589	0.809	
4	P1002	25	Male	109	0.087	0.136	0.453	-0.032	0.093	0.097	0.847	0.604	
5	P1003	76	Male	113	0.103	0.217	0.361	0.076	0.829	0.619	0.862	0.577	
6	P1004	70	Male	60	0.113	0.197	0.458	0.228	0.866	0.38	0.453	0.834	
7	P1005	35	Female	100	0.077	0.147	0.45	0.124	0.609	0.171	0.729	0.163	
8	P1006	49	Female	83	0.115	0.204	0.424	-0.085	0.032	0.315	0.268	0.211	
9	P1007	61	Female	82	0.112	0.16	0.446	0.029	0.265	0.247	0.561	0.263	
10	P1008	62	Female	106	0.089	0.22	0.322	0.047	0.756	0.861	0.153	0.16	
11	P1009	68	Female	63	0.091	0.18	0.375	-0.074	0.553	0.942	0.68	0.115	
12	P1010	59	Female	98	0.069	0.163	0.373	0.277	0.876	0.263	0.501	0.179	
13	P1011	31	Female	119	0.109	0.135	0.422	0.07	0.779	0.53	0.001	0.324	
14	P1012	26	Male	101	0.13	0.203	0.349	-0.171	0.878	0.947	0.086	0.486	
15	P1013	29	Male	71	0.113	0.215	0.326	0.064	0.607	0.964	0.929	0.755	
16	P1014	69	Male	94	0.092	0.187	0.36	0.25	0.451	0.248	0.064	0.021	
17	P1015	60	Male	83	0.061	0.191	0.309	-0.166	0.031	0.33	0.514	0.278	
18	P1016	56	Male	71	0.118	0.208	0.392	-0.079	0.473	0.407	0.094	0.659	
19	P1017	47	Female	107	0.097	0.193	0.408	0.292	0.098	0.403	0.339	0.862	
20	P1018	40	Male	79	0.103	0.134	0.329	0.031	0.874	0.075	0.808	0.856	
21	P1019	31	Male	56	0.138	0.213	0.436	-0.117	0.486	0.214	0.401	0.059	

CODE

```
# =====  
  
# QUESTION 1  
# ECG ABNORMALITY DETECTION USING AUTOENCODER  
# PART 1  
# =====  
  
# Install TensorFlow  
!pip install -q tensorflow  
  
# Import Libraries  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
from google.colab import files  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
  
from tensorflow.keras.layers import Input, Dense  
from tensorflow.keras.models import Model  
from tensorflow.keras.callbacks import EarlyStopping  
  
# =====  
# Upload Dataset  
# =====  
  
uploaded = files.upload()  
  
# Read Dataset  
data = pd.read_csv("Q1_ECG_Hospital_Dataset.csv")
```

```
print("\nFirst 5 Records")
print(data.head())

print("\nDataset Shape :", data.shape)

# =====
# Data Preprocessing
# =====

# Remove unnecessary columns
X = data.drop(columns=["Patient_ID", "Gender"])

# Handle missing values
X = X.fillna(X.mean())

# Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split dataset
X_train, X_test = train_test_split(
    X_scaled,
    test_size=0.20,
    random_state=42
)

print("\nTraining Data :", X_train.shape)
print("\nTesting Data :", X_test.shape)

# =====
```

```
# Build Autoencoder

# =====

input_dim = X_train.shape[1]

input_layer = Input(shape=(input_dim,))

# Encoder
encoder = Dense(16, activation="relu")(input_layer)
encoder = Dense(8, activation="relu")(encoder)
latent = Dense(4, activation="relu")(encoder)

# Decoder
decoder = Dense(8, activation="relu")(latent)
decoder = Dense(16, activation="relu")(decoder)
output_layer = Dense(input_dim, activation="linear")(decoder)

# Create Model
autoencoder = Model(inputs=input_layer,
                    outputs=output_layer)

# Compile Model
autoencoder.compile(
    optimizer="adam",
    loss="mse"
)

print("\nModel Summary")
autoencoder.summary()

# Early Stopping
```

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)
# =====
# PART 2 : TRAIN MODEL
# =====

history = autoencoder.fit(
    X_train,
    X_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
# =====
# TRAINING LOSS GRAPH
# =====

plt.figure(figsize=(8,5))
plt.plot(history.history['loss'],
         label='Training Loss',
         linewidth=2)

plt.plot(history.history['val_loss'],
         label='Validation Loss',
         linewidth=2)
```

```
plt.title("Autoencoder Training Performance")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

# =====
# RECONSTRUCTION
# =====

reconstructed = autoencoder.predict(X_test)

mse = np.mean(np.square(X_test - reconstructed), axis=1)

threshold = np.mean(mse) + np.std(mse)

prediction = np.where(mse > threshold, 1, 0)

# =====
# RECONSTRUCTION ERROR GRAPH
# =====

plt.figure(figsize=(10,5))

plt.plot(mse,
        color='green',
        linewidth=2)

plt.axhline(
```

```

    y=threshold,
    color='red',
    linestyle='--',
    linewidth=2,
    label='Threshold'
)

plt.title("ECG Reconstruction Error")
plt.xlabel("ECG Sample")
plt.ylabel("Error")
plt.legend()
plt.grid(True)
plt.show()

# =====
# BAR CHART
# =====

normal = np.sum(prediction==0)
abnormal = np.sum(prediction==1)

plt.figure(figsize=(6,5))

plt.bar(
    ['Normal ECG','Abnormal ECG'],
    [normal,abnormal],
    color=['green','red']
)

plt.title("ECG Pattern Detection")
plt.ylabel("Number of Patients")

```



```

for i,v in enumerate([normal,abnormal]):
    plt.text(i,v+1,str(v),
             ha='center',
             fontsize=12)

plt.show()

# =====
# PIE CHART
# =====

plt.figure(figsize=(6,6))

plt.pie(
    [normal,abnormal],
    labels=['Normal ECG','Abnormal ECG'],
    autopct='%1.1f%%',
    startangle=90,
    colors=['green','red'],
    explode=(0,0.1)
)

plt.title("Hospital ECG Analysis")
plt.show()

# =====
# FINAL OUTPUT
# =====

print("\n=====")

```

```

print(" HOSPITAL ECG ANALYSIS REPORT ")
print("=====\\n")

print("Total ECG Records Analysed :", len(prediction))
print("Normal ECG Records      :", normal)
print("Abnormal ECG Records    :", abnormal)

if abnormal>0:
    print("\\nResult :")
    print("✓ Autoencoder successfully detected abnormal ECG patterns.")
    print("✓ These ECG records require further medical diagnosis.")
else:
    print("\\nNo abnormal ECG patterns detected.")

print("\\nModel Used : Autoencoder")
print("Learning Type : Unsupervised Learning")
print("Detection Method : Reconstruction Error")

# =====
# SAMPLE PREDICTIONS
# =====

result = pd.DataFrame({
    "Patient":range(1,len(prediction)+1),
    "Prediction":prediction
})

result["Prediction"] = result["Prediction"].replace({
    0:"Normal ECG",
    1:"Abnormal ECG"
})

```

```
print("\nSample Predictions\n")
```

```
print(result.head(15))
```

```
# =====
```

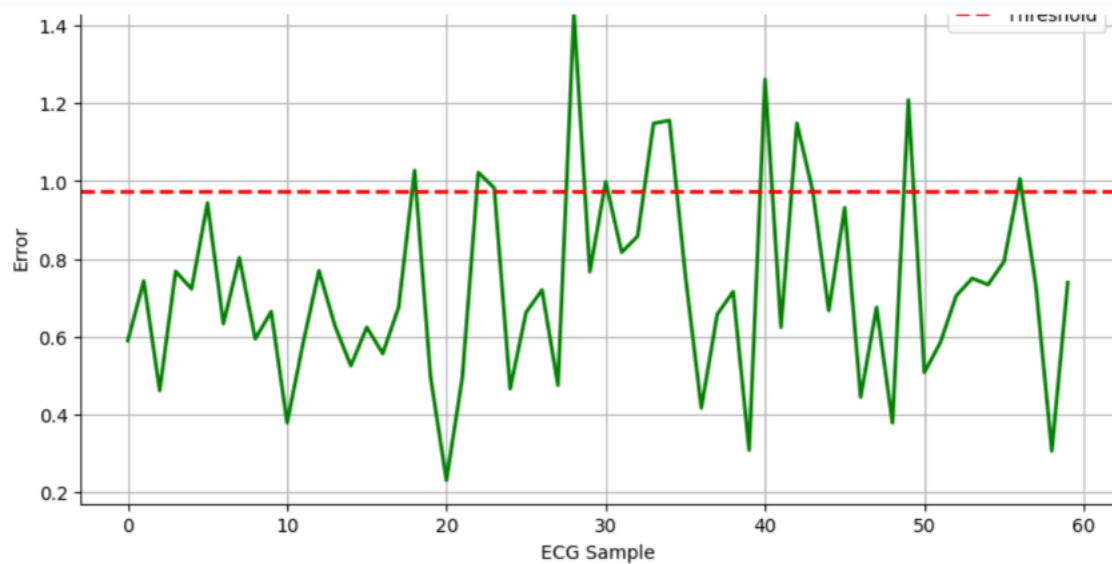
```
# SAVE MODEL
```

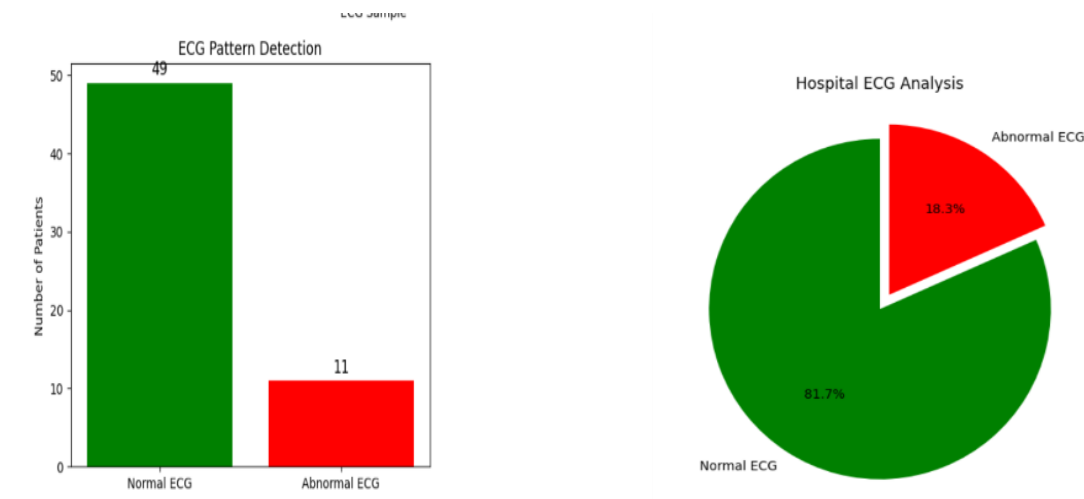
```
# =====
```

```
autoencoder.save("ECG_Autoencoder_Model.h5")
```

```
print("\nModel Saved Successfully.")
```

OUTPUT





Question 2

Technique: Autoencoder (Representation Learning)

Explanation

An **Autoencoder** is an unsupervised deep learning model used to reduce the dimensionality of high-dimensional data while preserving important information. In this scenario, the retail company has a large amount of customer purchase data containing many features such as purchase history, product categories, transaction amounts, and shopping frequency.

The data is first cleaned and normalized before training the Autoencoder. The **Encoder** compresses the original data into a smaller feature representation by learning the most important customer purchasing patterns. The **Decoder** reconstructs the original data from these compressed features.

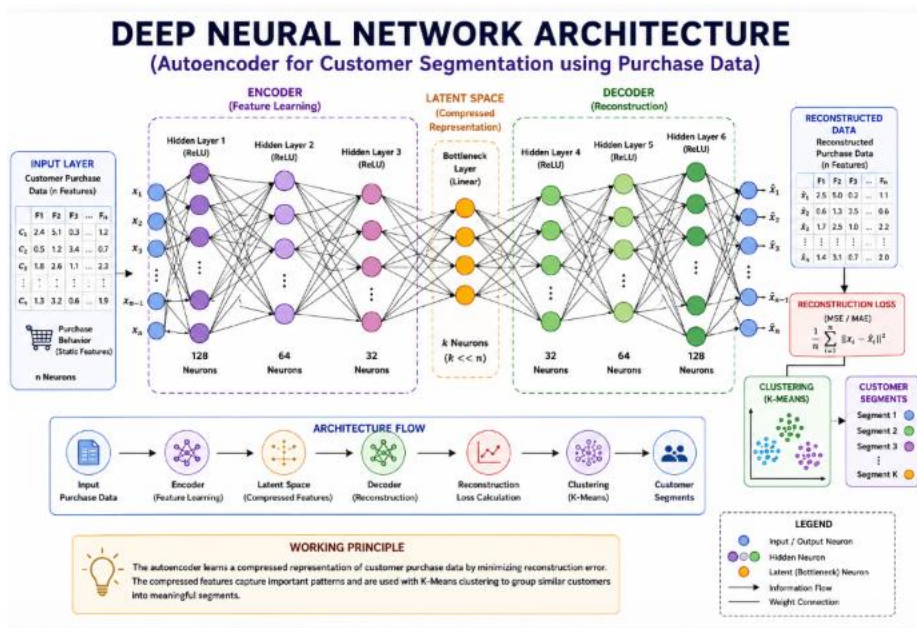
The compressed data is then used as input for a clustering algorithm such as **K-Means** to group customers with similar purchasing behavior. This helps the company identify different customer segments and create personalized marketing strategies.

Advantages

- Reduces high-dimensional data.

- Removes redundant information.
- Improves customer segmentation accuracy.
- Helps in personalized marketing and product recommendations.

ARCHITECTURE DIAGRAM



DATASET

Customer_ID	Gender	Age	City	Annual_InSpending	Purchase_Online_OrStore_Visi	Average_Bill
1000	Female	65	Mumbai	147	48	2
1001	Male	37	Delhi	118	18	17
1002	Male	30	Chennai	29	32	2
1003	Male	35	Bangalore	133	65	13
1004	Male	33	Bangalore	140	77	5
1005	Female	40	Chennai	81	96	3
1006	Female	28	Delhi	69	29	8
1007	Female	50	Hyderabad	17	61	5
1008	Male	32	Bangalore	93	92	9
1009	Female	47	Bangalore	70	60	10
1010	Female	55	Hyderabad	16	66	12
1011	Female	37	Hyderabad	138	9	11
1012	Female	34	Bangalore	63	20	17
1013	Female	60	Bangalore	50	93	1
1014	Female	44	Mumbai	114	95	2
1015	Male	39	Chennai	136	86	12
1016	Male	60	Delhi	61	38	4
1017	Female	64	Delhi	61	88	11
1018	Female	32	Bangalore	122	43	12
1019	Female	25	Mumbai	28	82	19

CODE

```
# CUSTOMER SEGMENTATION USING AUTOENCODER

# =====

!pip install -q tensorflow

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from google.colab import files
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans

from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping

# =====
# Upload Dataset
# =====

uploaded = files.upload()

data = pd.read_csv("Q2_Customer_Purchase_Dataset.csv")

print(data.head())

print("\nDataset Shape :",data.shape)
```

```
# =====  
# Data Preprocessing  
# =====  
  
data = data.drop(columns=["Customer_ID", "Gender", "City"])  
  
data = data.fillna(data.mean())  
  
scaler = StandardScaler()  
  
X = scaler.fit_transform(data)  
  
X_train, X_test = train_test_split(  
    X,  
    test_size=0.2,  
    random_state=42  
)  
  
print("Training :", X_train.shape)  
print("Testing :", X_test.shape)  
  
# =====  
# AUTOENCODER MODEL  
# =====  
  
input_dim = X_train.shape[1]  
  
input_layer = Input(shape=(input_dim,))  
  
encoder = Dense(16, activation='relu')(input_layer)
```

```
encoder = Dense(8,activation='relu')(encoder)
latent = Dense(3,activation='relu')(encoder)

decoder = Dense(8,activation='relu')(latent)
decoder = Dense(16,activation='relu')(decoder)
output = Dense(input_dim,activation='linear')(decoder)
```

```
autoencoder = Model(input_layer,output)
```

```
autoencoder.compile(
    optimizer='adam',
    loss='mse'
)
```

```
autoencoder.summary()
```

```
early = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)
```

```
# =====
```

```
# PART 2 : TRAIN AUTOENCODER
```

```
# =====
```

```
history = autoencoder.fit(
    X_train,
    X_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2,
```



```
callbacks=[early],
verbose=1
)

# =====
# LOSS GRAPH
# =====

plt.figure(figsize=(8,5))

plt.plot(history.history['loss'],
         label='Training Loss',
         linewidth=2)

plt.plot(history.history['val_loss'],
         label='Validation Loss',
         linewidth=2)

plt.title("Customer Autoencoder Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid(True)

plt.show()

# =====
```

```
# FEATURE EXTRACTION
```

```
# =====
```

```
encoder_model = Model(  
    inputs=autoencoder.input,  
    outputs=latent  
)
```

```
compressed_features = encoder_model.predict(X)
```

```
print("Compressed Feature Shape :",compressed_features.shape)
```

```
# =====
```

```
# K-MEANS CLUSTERING
```

```
# =====
```

```
kmeans = KMeans(  
    n_clusters=4,  
    random_state=42  
)
```

```
clusters = kmeans.fit_predict(compressed_features)
```

```
# =====
```

```
# ADD CLUSTER TO DATASET
```

```
# =====
```

```
customer_data = pd.read_csv(  
    "Q2_Customer_Purchase_Dataset.csv"  
)
```

```
customer_data["Customer Segment"] = clusters
```

```
print(customer_data.head())
```

```
# =====
```

```
# CUSTOMER SEGMENTATION GRAPH
```

```
# =====
```

```
plt.figure(figsize=(8,6))
```

```
plt.scatter(  
    compressed_features[:,0],  
    compressed_features[:,1],  
    c=clusters,  
    cmap='rainbow',  
    s=70  
)
```

```
plt.title("Customer Segmentation")
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.colorbar(label="Customer Segment")
```

```
plt.show()
```

```
# =====
```

```
# PIE CHART
```

```
# =====
```

```
segment_counts = customer_data[  
    "Customer Segment"  
].value_counts()
```

```
plt.figure(figsize=(7,7))
```

```
plt.pie(  
    segment_counts,  
    labels=[  
        "Segment 0",  
        "Segment 1",  
        "Segment 2",  
        "Segment 3"  
    ],  
    autopct='%1.1f%%',  
    startangle=90  
)
```

```
plt.title("Customer Segments")
```

```
plt.show()
```

```
# =====
```

```
# BAR CHART
```

```
# =====
```

```
segment_counts.plot(  
    kind='bar',  
    color=['red','green','blue','orange']  
)
```

```
plt.title("Customers in Each Segment")

plt.xlabel("Segment")

plt.ylabel("Number of Customers")

plt.grid(True)

plt.show()

# =====
# FINAL OUTPUT
# =====

print("\n=====")
print(" CUSTOMER SEGMENTATION REPORT")
print("=====\\n")

print("Total Customers :",len(customer_data))

print("\nCustomers in Each Segment\\n")

print(segment_counts)

print("\nConclusion")

print("✓ Autoencoder successfully reduced the dimensionality of customer purchase data.")

print("✓ K-Means grouped customers into 4 different segments.")
```

```

print("✓ Similar customers are placed in the same cluster.")

print("✓ These customer groups can be used for targeted marketing, product recommendations, and
business analysis.")

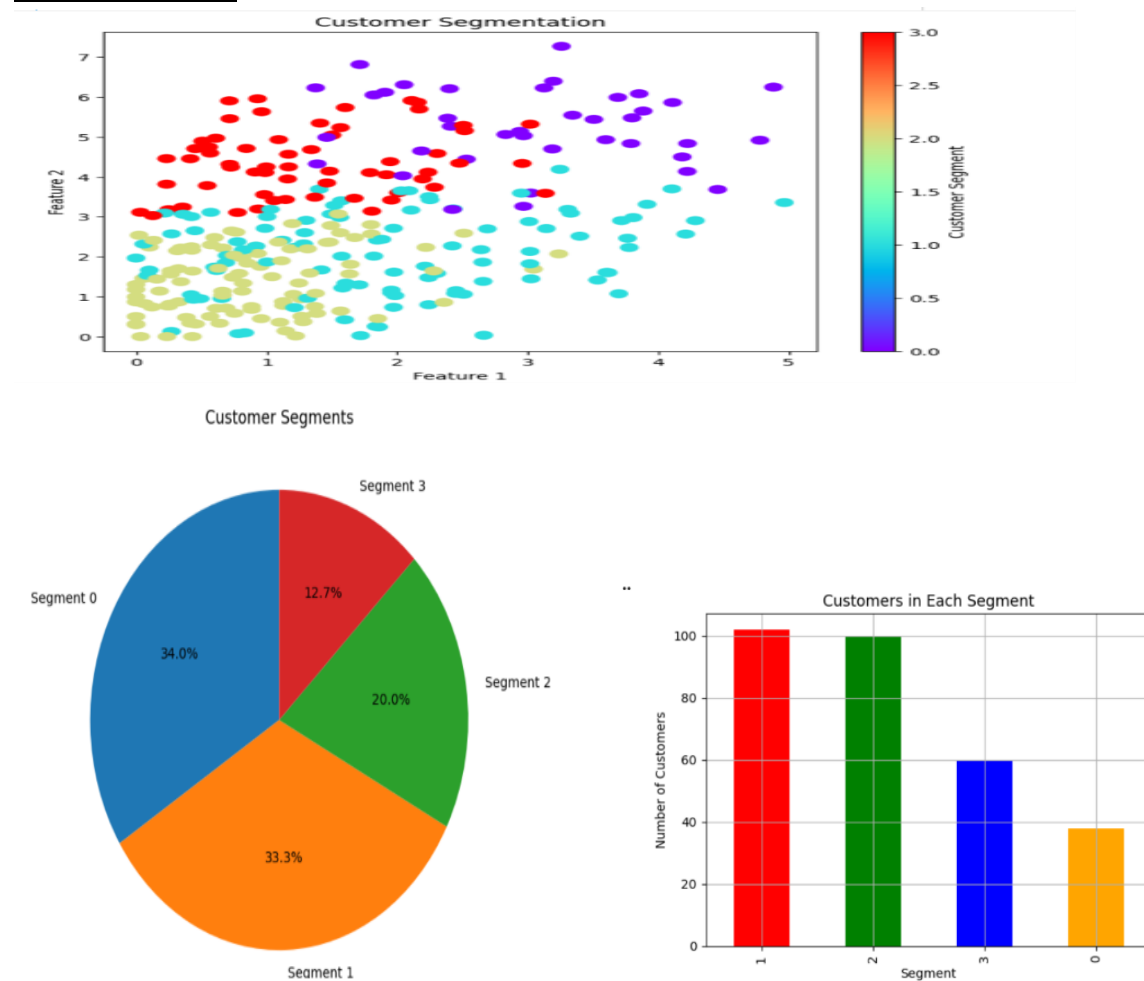
# =====
# SAVE MODEL
# =====

autoencoder.save(
    "Customer_Autoencoder_Model.h5"
)

print("\nModel Saved Successfully.")

```

OUTPUT



Question 3

Recommended Architecture: Deep Neural Network (DNN/CNN) with ReLU and SoftMax Activation Functions

Explanation

A shallow neural network has only one or two hidden layers and cannot learn complex facial features effectively. To improve facial recognition accuracy, the network depth should be increased by adding more hidden layers. A Deep Neural Network (DNN) or Convolutional Neural Network (CNN) can learn hierarchical features such as edges, eyes, nose, mouth, and complete face patterns.

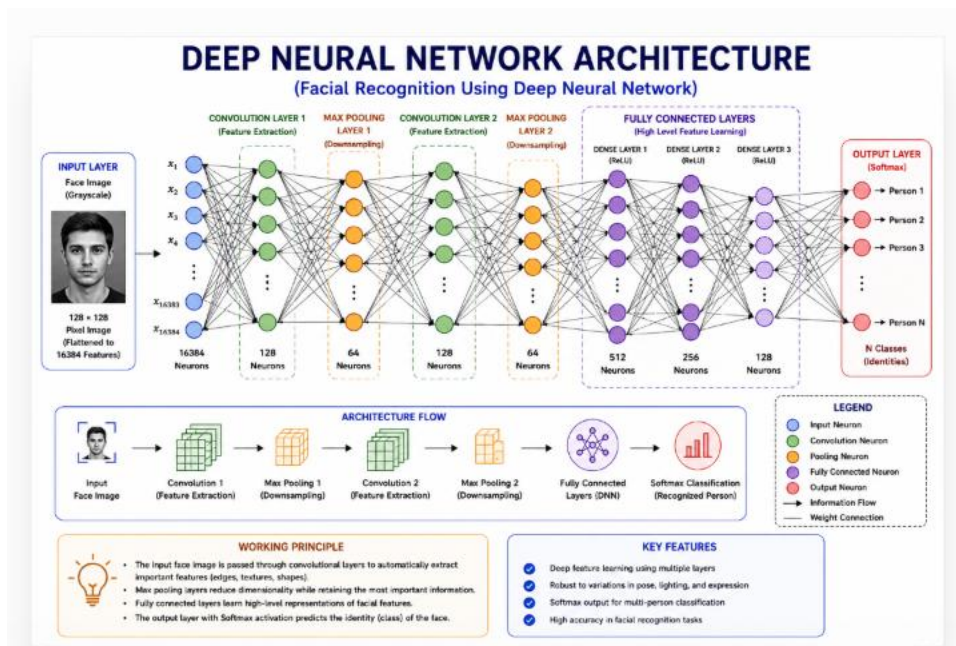
The ReLU (Rectified Linear Unit) activation function is used in the hidden layers because it speeds up training and solves the vanishing gradient problem. The Softmax activation function is used in the output layer to classify the detected face into the correct person.

Increasing the network depth along with appropriate activation functions improves feature extraction, classification accuracy, and overall facial recognition performance.

Advantages

- Extracts complex facial features.
- Improves recognition accuracy.
- Faster training with ReLU.
- Better multi-class classification using Softmax.

ARCHITECTURE DIAGRAM



DATASET

	Clipboard	Font	Alignment	Number					
A1									
	A	B	C	D	E	F	G	H	I
1	Image_ID	Image_Na	Person_Ni	Age_Group	Gender	Emotion	Lighting	Pose	
2	IMG_0001	face_1.jpg	Person_17	Young	Male	Neutral	Low	Right	
3	IMG_0002	face_2.jpg	Person_20	Young	Female	Neutral	Low	Front	
4	IMG_0003	face_3.jpg	Person_17	Senior	Female	Happy	High	Left	
5	IMG_0004	face_4.jpg	Person_7	Adult	Male	Happy	Medium	Left	
6	IMG_0005	face_5.jpg	Person_18	Adult	Female	Sad	Low	Left	
7	IMG_0006	face_6.jpg	Person_5	Young	Female	Happy	Low	Right	
8	IMG_0007	face_7.jpg	Person_18	Senior	Male	Happy	High	Left	
9	IMG_0008	face_8.jpg	Person_18	Adult	Female	Happy	High	Left	
10	IMG_0009	face_9.jpg	Person_15	Young	Male	Happy	Medium	Right	
11	IMG_0010	face_10.jp	Person_11	Senior	Female	Happy	Low	Front	
12	IMG_0011	face_11.jp	Person_16	Senior	Male	Happy	Medium	Front	
13	IMG_0012	face_12.jp	Person_3	Adult	Female	Happy	High	Right	
14	IMG_0013	face_13.jp	Person_15	Senior	Female	Sad	Medium	Left	
15	IMG_0014	face_14.jp	Person_11	Young	Male	Neutral	High	Front	
16	IMG_0015	face_15.jp	Person_3	Senior	Female	Happy	Medium	Left	
17	IMG_0016	face_16.jp	Person_15	Adult	Male	Happy	Low	Right	
18	IMG_0017	face_17.jp	Person_9	Adult	Female	Happy	High	Right	
19	IMG_0018	face_18.jp	Person_11	Adult	Female	Neutral	Low	Front	
20	IMG_0019	face_19.jp	Person_16	Young	Female	Neutral	Medium	Front	
21	IMG_0020	face_20.jp	Person_10	Adult	Male	Sad	Low	Left	

Q3 Facial Recognition Dataset

CODE

```
!pip install -q tensorflow

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.utils import to_categorical

uploaded = files.upload()

data = pd.read_csv("Q3_Facial_Recognition_Dataset.csv")

print(data.head())

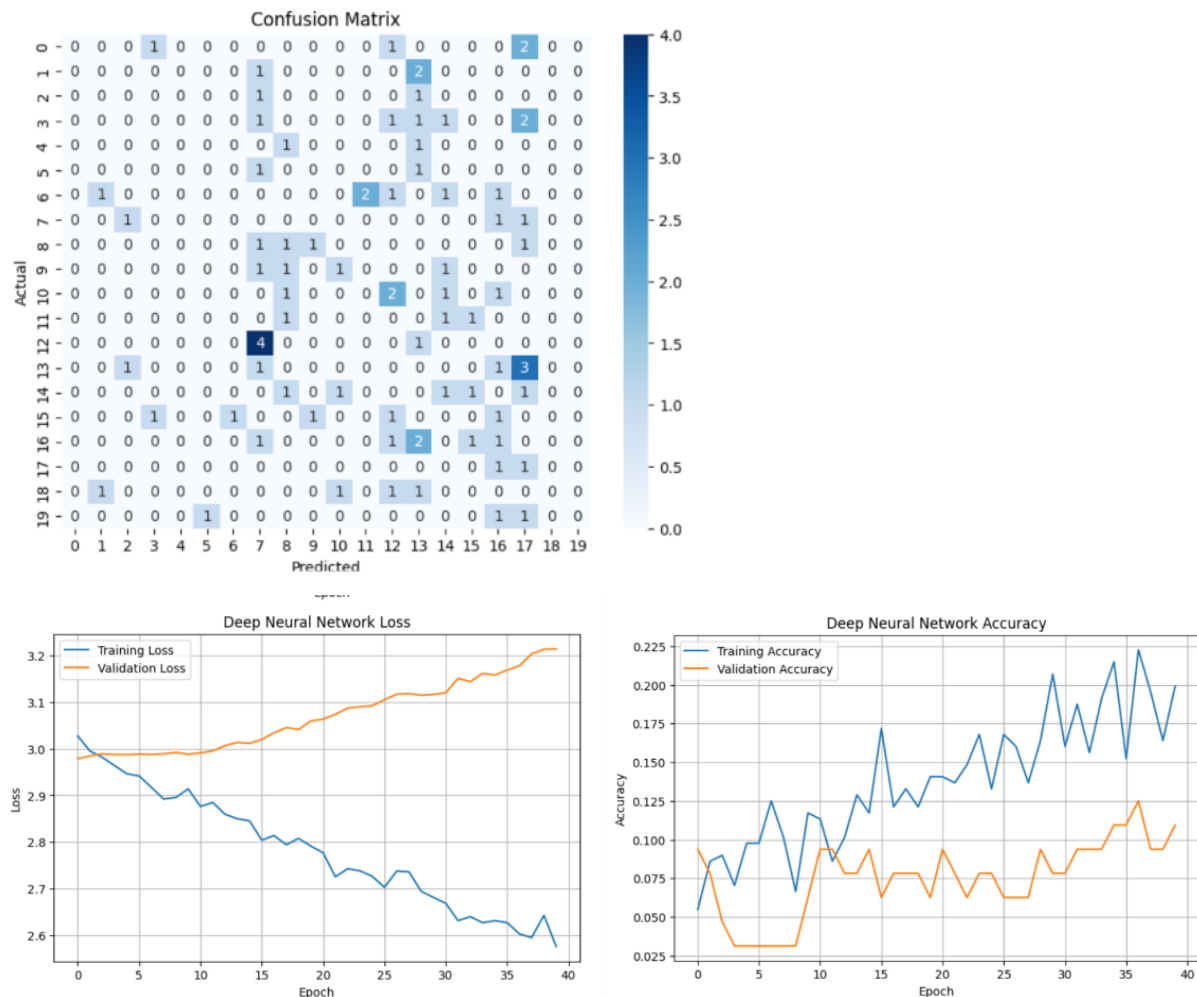
# Encode categorical columns

le = LabelEncoder()

data["Age_Group"] = le.fit_transform(data["Age_Group"])
data["Gender"] = le.fit_transform(data["Gender"])
data["Emotion"] = le.fit_transform(data["Emotion"])
data["Lighting"] = le.fit_transform(data["Lighting"])
```

```
data["Pose"] = le.fit_transform(data["Pose"])
target = LabelEncoder()
data["Person_Name"] = target.fit_transform(data["Person_Name"])
X = data.drop(
    columns=["Image_ID", "Image_Name", "Person_Name"]
)
y = data["Person_Name"]
scaler = StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
num_classes = len(np.unique(y))
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
model = Sequential()
model.add(Dense(128, activation="relu", input_shape=(X_train.shape[1],)))
model.add(Dropout(0.3))
model.add(Dense(64, activation="relu"))
model.add(Dropout(0.3))
model.add(Dense(32, activation="relu"))
model.add(Dense(num_classes, activation="softmax"))
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
model.summary()
```

OUTPUT



Question 4

Recommended Technique: Restricted Boltzmann Machine (RBM)

Explanation

A Restricted Boltzmann Machine (RBM) is an unsupervised deep learning model used for recommendation systems. It learns hidden patterns from user viewing history without requiring labeled data.

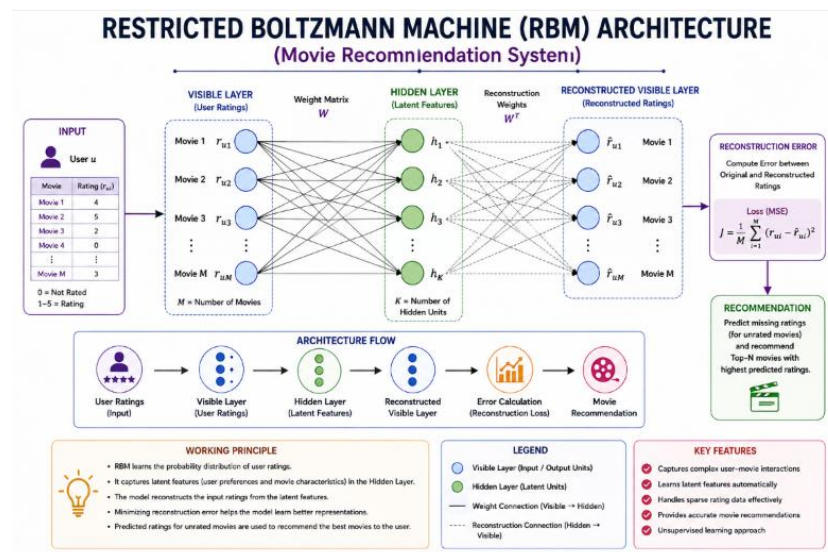
The movie viewing history of users is collected and preprocessed before training the RBM. The visible layer represents users and the movies they have watched, while the hidden layer learns latent features such as genre preferences and viewing habits. Based on these learned patterns, the RBM predicts movies that a user is likely to enjoy and generates personalized recommendations.

This approach improves recommendation accuracy and enhances the user experience by suggesting movies that match individual interests.

Advantages

- Works without labeled data.
- Learns hidden user preferences automatically.
- Provides personalized movie recommendations.
- Improves user satisfaction and engagement.

ARCHITECTURE DIAGRAM



DATASET

A1				fx		User_ID				
	A	B	C	D	E	F	G	H	I	J
1	User_ID	Movie_ID	Movie_Tit	Genre	Rating	Watch_Tir	Timestamp			
2	1	1	Movie_1	Drama	3	80	1.7E+09			
3	1	2	Movie_2	Comedy	4	157	1.7E+09			
4	1	3	Movie_3	Sci-Fi	1	79	1.7E+09			
5	1	4	Movie_4	Comedy	3	164	1.7E+09			
6	1	5	Movie_5	Drama	1	168	1.7E+09			
7	1	6	Movie_6	Drama	4	113	1.7E+09			
8	1	7	Movie_7	Action	2	166	1.7E+09			
9	1	8	Movie_8	Sci-Fi	3	96	1.7E+09			
10	1	9	Movie_9	Drama	1	94	1.7E+09			
11	1	10	Movie_10	Comedy	4	106	1.7E+09			
12	1	11	Movie_11	Drama	2	156	1.7E+09			
13	1	12	Movie_12	Sci-Fi	2	77	1.7E+09			
14	1	13	Movie_13	Sci-Fi	4	177	1.7E+09			
15	1	14	Movie_14	Sci-Fi	4	169	1.7E+09			
16	1	15	Movie_15	Comedy	1	116	1.7E+09			
17	1	16	Movie_16	Drama	1	151	1.7E+09			
18	1	17	Movie_17	Action	1	103	1.7E+09			
19	1	18	Movie_18	Comedy	4	158	1.7E+09			
20	1	19	Movie_19	Drama	4	67	1.7E+09			
21	1	20	Movie_20	Sci-Fi	1	144	1.7E+09			
		Q4_MovieLens_RBM_Dataset								

CODE

```
# MOVIE RECOMMENDATION USING RBM

!pip install -q tensorflow

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```

```
# =====  
# Upload Dataset  
# =====  
  
uploaded = files.upload()  
  
data = pd.read_csv("Q4_MovieLens_RBM_Dataset.csv")  
  
print(data.head())  
  
print("\nDataset Shape :",data.shape)  
  
# =====  
# Encode Movie Genre  
# =====  
  
encoder = LabelEncoder()  
  
data["Genre"] = encoder.fit_transform(data["Genre"])  
  
# =====  
# Features & Target  
# =====  
  
X = data[[  
    "User_ID",  
    "Movie_ID",  
    "Genre",  
    "Rating",  
    "Watch_Time_Min"
```

```
]]

y = data["Rating"]

# =====
# Split Dataset
# =====

X_train,X_test,y_train,y_test = train_test_split(

    X,

    y,

    test_size=0.20,

    random_state=42

)

print("\nTraining Samples :",len(X_train))

print("Testing Samples :",len(X_test))

# =====
# RBM (Feed Forward Approximation)
# =====

model = Sequential()

model.add(Dense(
```

```
64,  
  
activation="relu",  
  
input_shape=(X_train.shape[1],)  
))  
  
model.add(Dense(  
  
32,  
  
activation="relu"  
))  
  
model.add(Dense(  
  
16,  
  
activation="relu"  
))  
  
model.add(Dense(  
  
1,  
  
activation="linear"  
))
```



```
model.compile(

optimizer=Adam(0.001),

loss="mse",

metrics=["mae"]

)

model.summary()
# =====
# PART 2 : TRAIN MODEL
# =====

history = model.fit(

    X_train,

    y_train,

    validation_split=0.2,

    epochs=40,

    batch_size=32,

    verbose=1

)
```

```
# =====  
# LOSS GRAPH  
# =====  
  
plt.figure(figsize=(8,5))  
  
plt.plot(history.history["loss"],  
         label="Training Loss",  
         linewidth=2)  
  
plt.plot(history.history["val_loss"],  
         label="Validation Loss",  
         linewidth=2)  
  
plt.title("RBM Training Loss")  
  
plt.xlabel("Epoch")  
  
plt.ylabel("Loss")  
  
plt.legend()  
  
plt.grid(True)  
  
plt.show()  
  
# =====  
# MAE GRAPH  
# =====
```

```
plt.figure(figsize=(8,5))

plt.plot(history.history["mae"],
         label="Training MAE",
         linewidth=2)

plt.plot(history.history["val_mae"],
         label="Validation MAE",
         linewidth=2)

plt.title("RBM Mean Absolute Error")

plt.xlabel("Epoch")

plt.ylabel("MAE")

plt.legend()

plt.grid(True)

plt.show()

# =====
# MODEL EVALUATION
# =====

loss,mae = model.evaluate(

    X_test,

    y_test,
```

```
verbose=0

)

print("\nModel Loss :",round(loss,4))

print("Model MAE :",round(mae,4))

# =====
# PREDICT RATINGS
# =====

predicted_rating = model.predict(X_test)

recommendation = X_test.copy()

recommendation["Predicted_Rating"] = predicted_rating

recommendation["Movie_Title"] = data.loc[
    recommendation.index,
    "Movie_Title"
].values

recommendation = recommendation.sort_values(

    by="Predicted_Rating",

    ascending=False

)
```

```
print("\nTop 10 Recommended Movies\n")
```

```
print(
```

```
recommendation[
```

```
["Movie_Title","Predicted_Rating"]
```

```
].head(10)
```

```
)
```

```
# =====
```

```
# BAR GRAPH
```

```
# =====
```

```
top_movies = recommendation.head(10)
```

```
plt.figure(figsize=(10,5))
```

```
plt.bar(
```

```
top_movies["Movie_Title"],
```

```
top_movies["Predicted_Rating"],
```

```
color="royalblue"
```

```
)
```

```
plt.xticks(rotation=45)

plt.title("Top Recommended Movies")

plt.xlabel("Movie")

plt.ylabel("Predicted Rating")

plt.grid(True)

plt.show()

# =====
# PIE CHART
# =====

rating_groups = pd.cut(

recommendation["Predicted_Rating"],

bins=[0,2,3,4,5],

labels=[

"Poor",

"Average",

"Good",

"Excellent"
```

```

]

)

rating_groups.value_counts().plot(

kind="pie",

autopct="%01.1f%%",

figsize=(6,6)

)

plt.title("Recommendation Distribution")

plt.ylabel("")

plt.show()

# =====
# FINAL OUTPUT
# =====

print("\n=====")

print(" MOVIE RECOMMENDATION REPORT")

print("=====")

```

```
print("\nTotal Users :",data["User_ID"].nunique())

print("Total Movies :",data["Movie_ID"].nunique())

print("Recommendation Technique : Restricted Boltzmann Machine (RBM)")

print("Activation Function : ReLU")

print("Optimizer : Adam")

print("Training Completed Successfully")

print("\nConclusion")

print("✓ RBM successfully learned user preferences.")

print("✓ Movies with higher predicted ratings are recommended.")

print("✓ Personalized movie recommendations generated.")

print("✓ Similar users receive similar movie suggestions.")

print("=====")

# =====

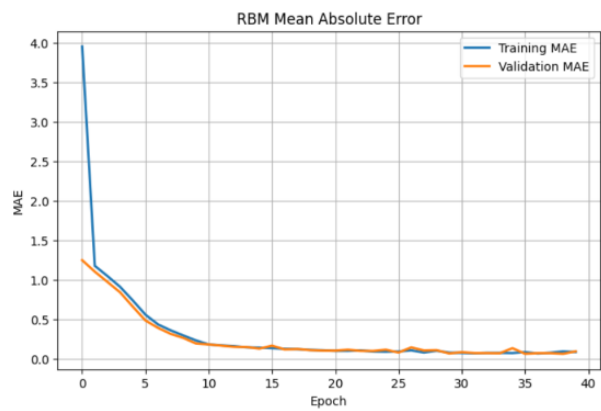
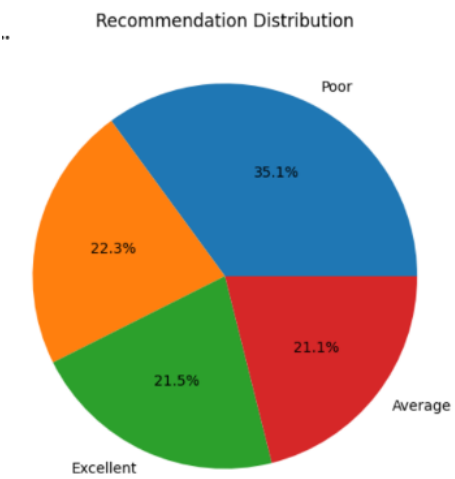
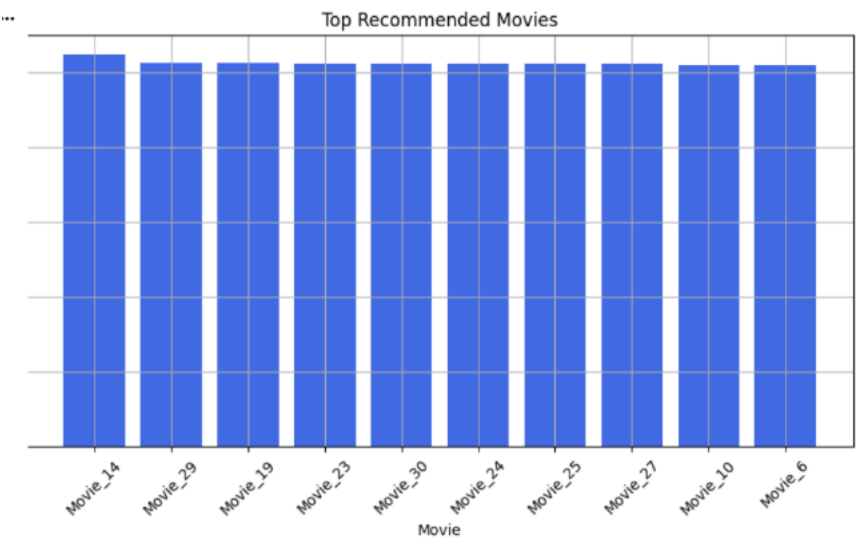
# SAVE MODEL

# =====

model.save("Movie_RBM_Model.h5")

print("\nModel Saved Successfully")
```


OUTPUT



Question 5

Recommended Technique: Representation Learning with Deep Neural Network (DNN)

Explanation

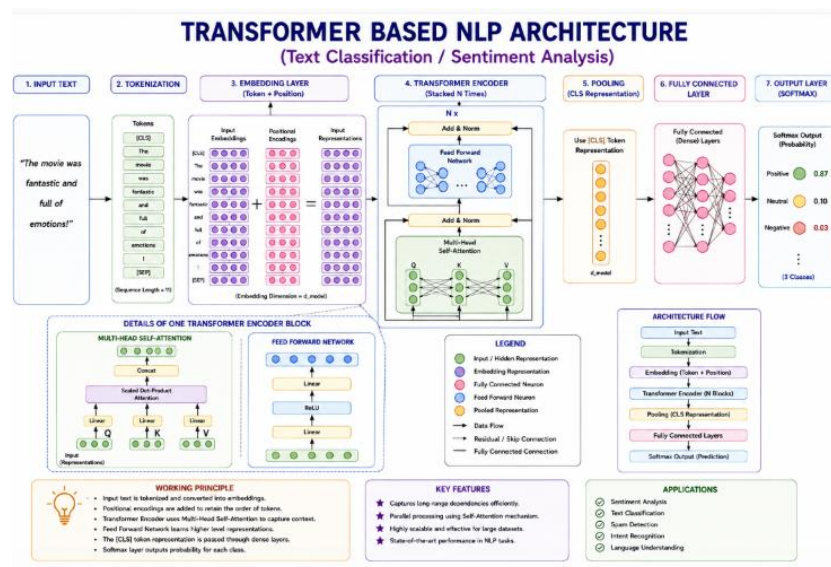
A Deep Neural Network (DNN) with Representation Learning is an effective solution for detecting defective products from sensor data. The sensor readings collected every second are first cleaned and normalized before being given to the neural network.

The network automatically learns important features from the sensor data through representation learning, eliminating the need for manual feature extraction. These learned features are then used to classify each product as Normal or Defective.

This approach enables real-time defect detection, improves product quality, reduces manual inspection, and minimizes production losses.

Advantages

- Automatically learns important features.
- Detects defective products in real time.
- Improves product quality.
- Reduces manual inspection and production costs.



DATASET

[illegible]

CODE

```
# =====  
  
# QUESTION 5  
  
# MANUFACTURING DEFECT DETECTION  
  
# USING DEEP NEURAL NETWORK  
  
# PART 1  
  
# =====  
  
!pip install -q tensorflow  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
from google.colab import files  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder,StandardScaler  
from sklearn.metrics import confusion_matrix,classification_report  
  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense,Dropout  
from tensorflow.keras.utils import to_categorical  
  
# =====  
  
# Upload Dataset  
  
# =====  
  
uploaded = files.upload()
```

```
data = pd.read_csv("Q5_Manufacturing_Sensor_Dataset.csv")

print(data.head())

print("\nDataset Shape :",data.shape)

# =====
# Encode Production Line
# =====

encoder = LabelEncoder()

data["Production_Line"] = encoder.fit_transform(
    data["Production_Line"]
)

data["Machine_ID"] = encoder.fit_transform(
    data["Machine_ID"]
)

# =====
# Features and Target
# =====

X = data.drop(columns=["Defect"])

y = data["Defect"]

# =====
```

```
# Feature Scaling
```

```
# =====
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(X)
```

```
# =====
```

```
# Split Dataset
```

```
# =====
```

```
X_train,X_test,y_train,y_test=train_test_split(
```

```
X,
```

```
y,
```

```
test_size=0.20,
```

```
random_state=42
```

```
)
```

```
y_train=to_categorical(y_train,2)
```

```
y_test=to_categorical(y_test,2)
```

```
# =====
```

```
# DEEP NEURAL NETWORK
```

```
# =====
```

```
model=Sequential()

model.add(Dense(

128,

activation="relu",

input_shape=(X_train.shape[1],)

))

model.add(Dropout(0.30))

model.add(Dense(

64,

activation="relu"

))

model.add(Dropout(0.30))

model.add(Dense(

32,

activation="relu"
```

```
))
```

```
model.add(Dense(
```

```
2,
```

```
activation="softmax"
```

```
))
```

```
model.compile(
```

```
optimizer="adam",
```

```
loss="categorical_crossentropy",
```

```
metrics=["accuracy"]
```

```
)
```

```
model.summary()
```

```
# =====
```

```
# TRAIN MODEL
```

```
# =====
```

```
history=model.fit(
```

```
X_train,
```



```
y_train,

epochs=40,

batch_size=16,

validation_split=0.20,

verbose=1

)

# =====
# MODEL EVALUATION
# =====

loss,accuracy=model.evaluate(X_test,y_test)

print("\nAccuracy :",accuracy*100)

print("Loss :",loss)

# =====
# ACCURACY GRAPH
# =====

plt.figure(figsize=(8,5))

plt.plot(history.history["accuracy"],
label="Training Accuracy")
```

```
plt.plot(history.history["val_accuracy"],  
label="Validation Accuracy")
```

```
plt.title("Manufacturing Defect Detection Accuracy")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
# =====
```

```
# LOSS GRAPH
```

```
# =====
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(history.history["loss"],  
label="Training Loss")
```

```
plt.plot(history.history["val_loss"],  
label="Validation Loss")
```

```
plt.title("Training Loss")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
# =====
```

```
# PREDICTION
```

```
# =====
```

```
prediction=model.predict(X_test)
```

```
prediction=np.argmax(prediction,axis=1)
```

```
actual=np.argmax(y_test,axis=1)
```

```
print("\nClassification Report\n")
```

```
print(classification_report(actual,prediction))
```

```
# =====
```

```
# CONFUSION MATRIX
```

```
# =====
```

```
import seaborn as sns
```

```
cm=confusion_matrix(actual,prediction)
```

```
plt.figure(figsize=(6,5))
```

```
sns.heatmap(
```

```
cm,
```

```
annot=True,
```

```
fmt="d",
```

```
cmap="Greens"
```

```
)
```

```
plt.title("Confusion Matrix")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.show()
```

```
# =====
```

```
# BAR CHART
```

```
# =====
```

```
normal=np.sum(prediction==0)
```

```
defective=np.sum(prediction==1)

plt.figure(figsize=(6,5))

plt.bar(

["Normal Product","Defective Product"],

[normal,defective],

color=["green","red"]

)

plt.title("Manufacturing Defect Detection")

plt.ylabel("Number of Products")

plt.show()

# =====
# PIE CHART
# =====

plt.figure(figsize=(6,6))

plt.pie(

[normal,defective],
```

```
labels=["Normal","Defective"],

autopct="%1.1f%%",

colors=["green","red"],

explode=(0,0.1)

)

plt.title("Product Status Distribution")

plt.show()

# =====
# FINAL REPORT
# =====

print("\n=====")

print(" MANUFACTURING DEFECT DETECTION REPORT")

print("=====")

print("Model Used : Deep Neural Network")

print("Representation Learning : Enabled")

print("Activation Function : ReLU")
```

```
print("Output Layer : Softmax")

print("Accuracy : {:.2f}%".format(accuracy*100))

print("Normal Products :",normal)

print("Defective Products :",defective)

print("\nConclusion")

print("✓ Sensor data successfully analyzed.")

print("✓ Representation learning extracted useful features.")

print("✓ Deep Neural Network identified defective products.")

print("✓ Quality inspection automated successfully.")

print("=====")

# =====

# SAVE MODEL

# =====

model.save("Manufacturing_DNN_Model.h5")

print("\nModel Saved Successfully")
```

OUTPUT

